

ROS 2 and Gazebo Harmonic Sensor Integration

Technical Implementation Report

Student: Sina Jovy | A01833561

Campus: Puebla

Course: Intelligent Mobility

Module: 4: Control | Activity 1

GitHub Repository

https://github.com/a01833561-ship-it/Ackermann_vehicle

September 2026

1 Introduction

The aim of this project is a simulation of the QCar using ROS 2, RViz, and Gazebo. The sensors should deliver a realistic output. The simulation should be able to launch from a singular input to the Ubuntu Terminal.

Software environment

The **Robot Operating System (ROS)** is a set of open source software libraries and tools.

It is used for robot software communication. With the Lab Computer the version Jazzy Jalisco was used - compatible for Ubuntu 24.04.

Gazebo Sim, version 8.11.0 is a 3D simulator - It computes physics simulation, a suite of sensors, and interfaces for programs and GUI.

RViz 2 is a 3D visualizer for the ROS 2 framework, used for visualization and verification of sensor data.

xacro is a preprocessor tool in python. It is used to simplify the generation of URDF files - with macro shortcuts that automate a series of steps. The use of Xacro modules keeps the sensor implementation independent from the base vehicle description.

Required **ROS packages** :

```
ros_gz_sim
ros_gz_bridge
robot_state_publisher
joint_state_publisher_gui
rviz2
teleop_twist_keyboard
```

2 System Architecture

ROS 2 Workspace Structure

- **URDF/ xacro model:** `mobile_robot_base.urdf.xacro` Defines the vehicle's physical links and joint kinematics. The `urdf` folder contains the xacro macros of the sensors. `mobile_robot_base.urdf.xacro` declares base geometry, includes sensors Xacro macros `mobile_robot_gazebo.xacro` - gazebo references for RViz configuration
- **Gazebo Plugins :** For the `gz::sim::systems::AckermannSteering` drive controller and simulation properties, GPU LiDAR rendering, camera plugins.
- **State Publisher:** Is a plugin in `mobile_robot_gazebo.xacro`. It reads joint states and publishes the official TF tree for coordinate transformations across the robot.
- **ackermann26_vehicle_gazebo/:** Contains the Gazebo simulation launch file and the ROS-Gazebo bridge configuration.

Directory Tree Layout

```
sm26_ws_new/  
src/  
  ackermann26_vehicle_description/  
    urdf/  
      common_properties.xacro  
      imu_sensor.xacro  
      lidar.xacro  
      mobile_robot_base.urdf.xacro  
      mobile_robot_gazebo.xacro  
      rgb_camera.xacro  
    launch/  
      display.launch.py  
    rviz/  
      config.rviz  
    package.xml  
    CMakeLists.txt  
  
  ackermann26_vehicle_gazebo/  
    launch/  
      gz_sim_launch.py  
    config/  
      ros_bridge.yaml  
    package.xml  
    CMakeLists.txt  
  
build/  
install/  
log/
```

Sensors and cameras send information through Topics: `/lidar_scan`, `/front_camera/image_raw` etc. `/rgb_camera`, `/imu/data`.
topic info can contain: Orientation Angular Vel. Linear Acc.

The build, install, and log directories are generated by the ROS 2 build system.

3 Communication and Sensors

"Gazebo provides the simulated sensor data, `ros_gz_bridge` transfers the data to ROS 2, and TF ensures that all sensor measurements are associated with the correct coordinate frames."

The Bridge

`ros_gz_bridge` is for communication between Gazebo Transport and ROS 2.

TF stands for the TF2 Transform Library - It manages the coordinate systems of individual parts (links) connected by joints using a tree-like hierarchy.

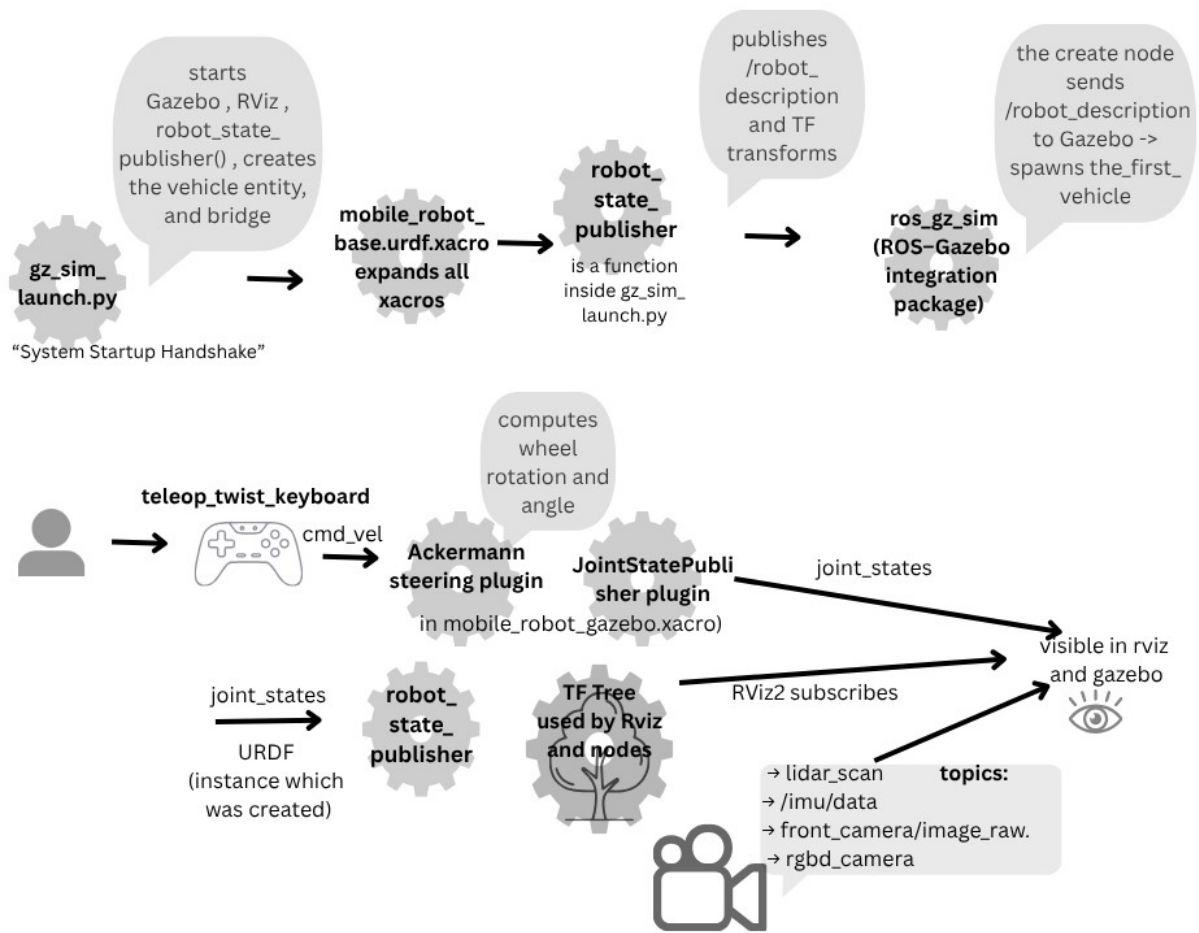


Figure 1: Communication Loop, beginning from Launch

4 User Instructions

Source the workspace and build package

```
1 cd Workspaces/sm26_ws_new/
2 source install/setup.bash
3 colcon build --symlink-install
```

Starting a Gazebo world - select Empty and Launch

```
1 gz sim
2 ros2 launch ackermann26_vehicle_gazebo gz_sim_launch.py
```

or for only RViz vizualization:

```
1 ros2 launch ackermann26_vehicle_description display.launch.py
```

Keyboard control needs a sepearte terminal command

```
1 ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Keyboard control with
U I O
J K L
M < >

End simulation with Ctrl + C

References

- [1] Open Robotics, *SDFFormat Specification*. Available at: <https://sdformat.org/spec>
- [2] Open Robotics, *Gazebo Documentation*. Available at: <https://gazebo.org/docs/>
- [3] Open Robotics, *ros_gz_bridge Documentation*. Available at: <https://github.com/gazebo/rosgz>
- [4] Open Robotics, *ROS 2 Documentation*. Available at: <https://docs.ros.org/>
- [5] Michael Behrens *xacro documentation*. Available at: <https://github.com/ros/xacro/wiki>
- [6] Daniel Sosa *University Course Project Folder*. Available at: https://github.com/dsosa114/movilidad_inteligente

AI Declaration

Generative AI was used as a programming copilot to troubleshoot ROS 2 launch scripts, configure YAML bridge mappings, and debug Xacro/XML macro syntax. The chat panel integrated in Visual Studio Code uses GitHub Copilot, underlying AI models are autoselected. Prompts were used for identifying spelling errors and assisting with test runs. Some example prompts included “Test this file” and “add the collisions to the *camera_link* macro”.